High Level Design

For

Deployment of Education App to Kubernetes

Course Name: DevOps

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Sr.no	Student Name	Enrolment Number
1	ABHIKET THAKUR	EN22CS301030
2	ADITYA PALIWAL	EN22CS301055
3	AARUSHI MITTAL	EN24CS3T10026
4	AKASH KUSHWAHA	EN22CS301083
5	AKSHAY DASHORE	EN22CS301096
6	AKSHITA RATHORE	EN22CS301100

Institution Name: Medi-caps University – Datagami Skill Based Course

Group: 04D1

Project No: GAI-04

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Ritesh Joshi

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

Jan - May 2026

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document provides an overview of the architecture and design of the Education application deployed on a Kubernetes cluster. It outlines the major system components, their interactions, and the overall workflow involved in deploying and managing the application.

The scope includes the design of key modules such as the React frontend, Docker-based containerization, Kubernetes orchestration components (Deployments, Pods, and Services), and the CI/CD pipeline used for automated build and deployment. It also highlights the strategy for achieving zero-downtime deployments through rolling updates.

This document focuses on the high-level architecture and does not go into low-level implementation details such as code structure or internal algorithms. It is intended to provide a clear understanding of the system design, component relationships, and deployment flow.

1.2 Intended Audience

The primary audience includes:

- Project Evaluators and Academic Reviewers
- Development Team Members
- Future Maintainers and Enhancers
- Research-Oriented Audience

1.3 System overview

The system is designed to deploy and manage a containerized Education web application using Kubernetes. It follows a modern DevOps architecture that integrates containerization, orchestration, and continuous deployment to ensure scalability, reliability, and high availability. The core of the system is a React-based Education application that is packaged into a Docker container. This containerized application is then deployed onto a Kubernetes cluster using Minikube, which acts as a local testing environment for orchestration. Kubernetes manages the application through its key components such as Deployments and Services.

The Deployment ensures that a specified number of application instances (pods) are always running, providing fault tolerance and self-healing capabilities. The Service component exposes the application to users and distributes incoming traffic evenly across all running pods, ensuring efficient load balancing. Medicaps University – Datagami Skill Based Course – Project Report A significant feature of the system is the implementation of zero-downtime rolling updates. Whenever a new version of the application is released, Kubernetes gradually replaces old pods with new ones without interrupting the availability of the application.

This ensures a seamless user experience even during updates. The system also incorporates a CI/CD pipeline using tools like GitHub Actions or Jenkins. This pipeline automates the process of building the Docker image, pushing it to a container registry, and deploying it to the Kubernetes cluster. This reduces manual intervention and improves deployment efficiency. Overall, the system demonstrates how modern tools and technologies can be combined to create a scalable, automated, and resilient application deployment environment.

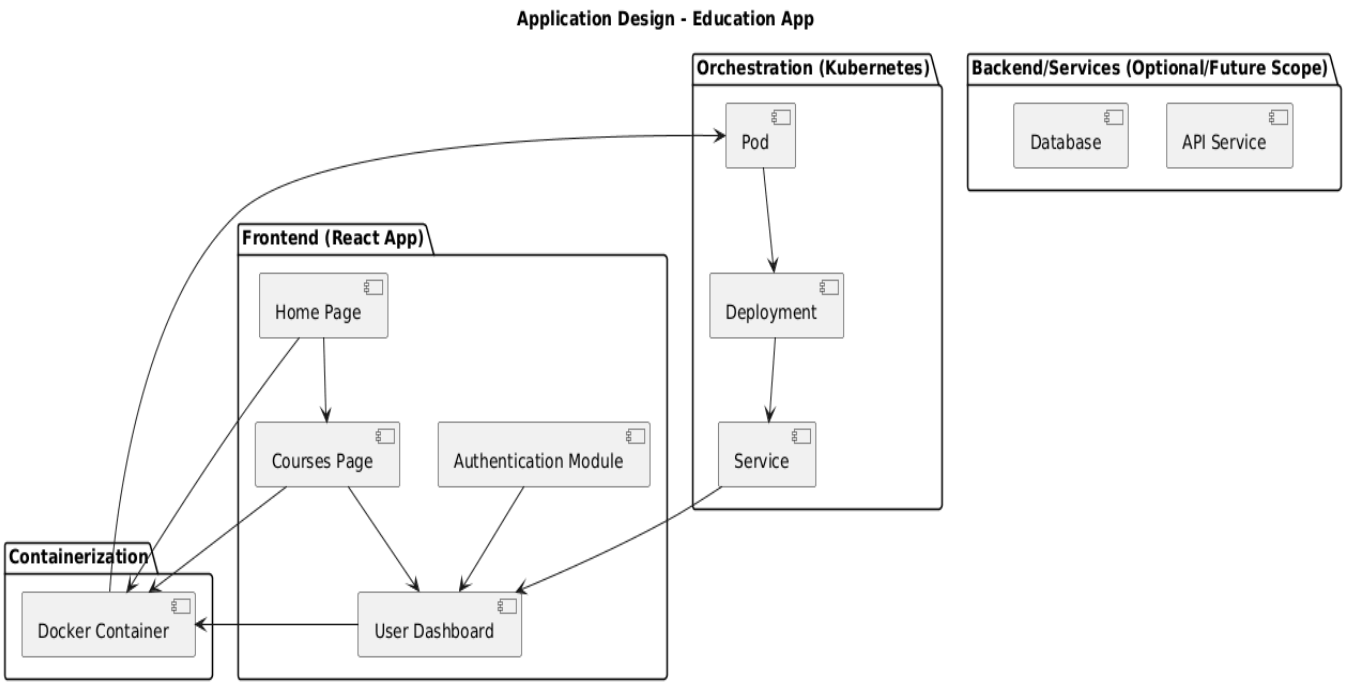
2. System Design

In this project, the system design focuses on deploying a containerized Education application using Kubernetes. It involves designing how different components—such as the React frontend, Docker containers, Kubernetes deployments, and services—interact with each other. The design ensures high availability, fault tolerance, and smooth application updates through features like rolling deployments. By incorporating modern DevOps practices, the system design also includes automation through CI/CD pipelines, enabling continuous integration and seamless delivery of application updates.

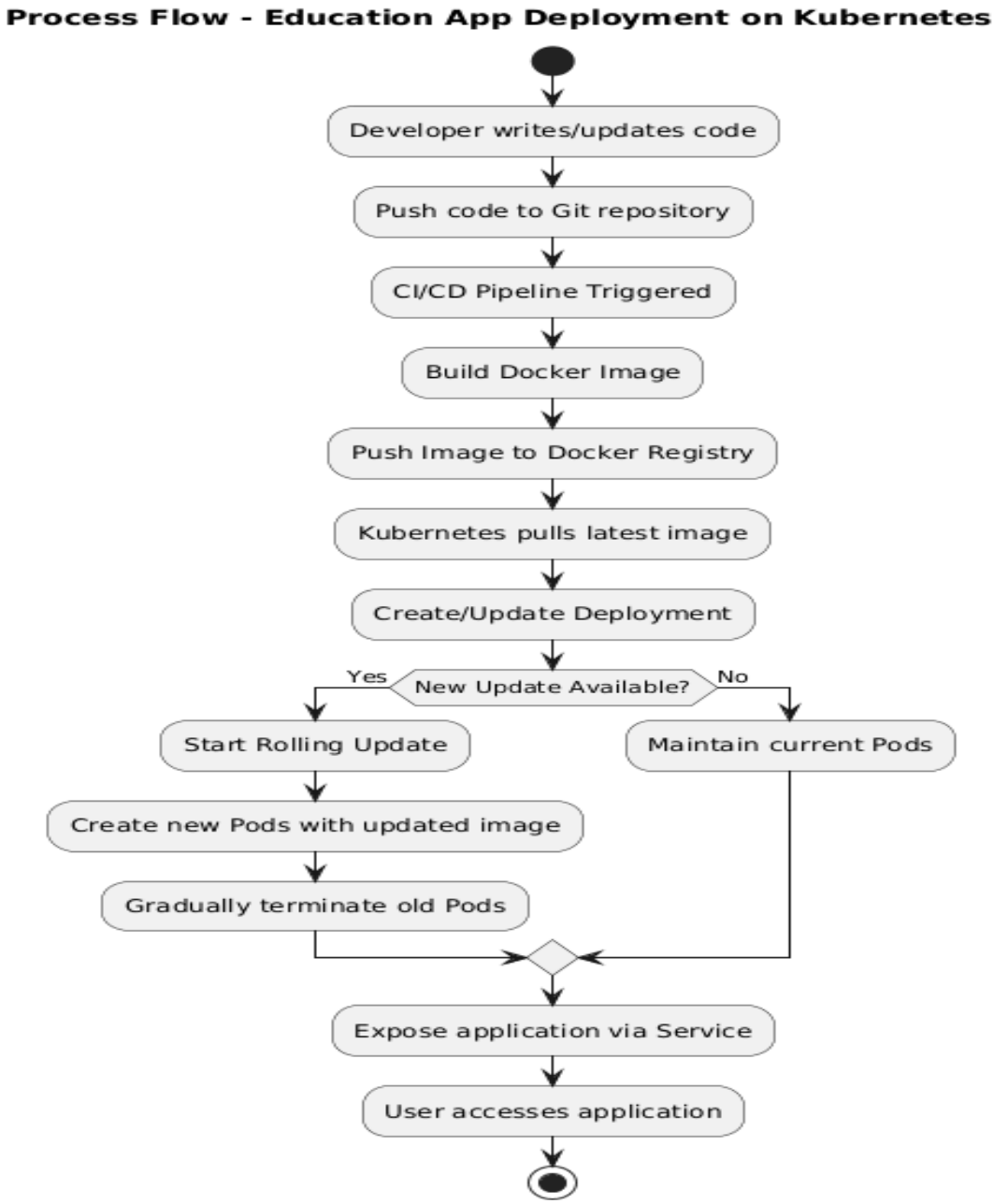
2.1 Application Design

Application design focuses on the structure and behavior of the application, including how its components are organized and interact with each other. It defines the user interface, functionality, and internal logic required to meet user needs.

In this project, the application design involves building a React-based Education app with a modular and responsive interface. The design ensures smooth user interaction, efficient data handling, and compatibility with containerization and deployment processes. It also considers performance, scalability, and ease of integration with Kubernetes-based deployment.



2.2 Process Flow



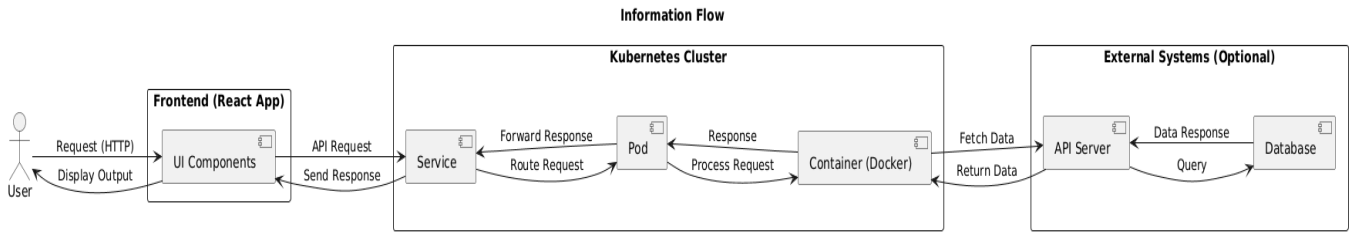
The process flow describes the sequence of steps involved in developing, containerizing, and deploying the Education application onto a Kubernetes cluster. It illustrates how the system moves from code development to continuous integration, container creation, and automated deployment.

The flow also highlights the role of CI/CD pipelines in streamlining the build and deployment process, as well as Kubernetes in managing application updates through rolling deployments. This ensures that the application remains available to users while updates are applied seamlessly.

2.3 Information Flow

The information flow represents how data moves within the system between different components during operation. It describes the exchange of information from the user to the application, through the deployed services, and back to the user.

In this project, information flows from the user interface of the React application to the containerized environment managed by Kubernetes, where requests are handled by pods and routed through services. This structured flow ensures efficient communication, proper request handling, and a seamless user experience.



2.4 Components Design

Component design defines the individual modules of the system and their interactions to achieve the overall functionality. It focuses on breaking down the system into manageable and reusable components, each responsible for a specific task.

In this project, the component design includes key elements such as the React frontend, Docker containers, CI/CD pipeline, and Kubernetes resources like Deployments, Pods, and Services. These components work together to ensure efficient application deployment, scalability, and smooth communication within the system.

2.5 Key Design Considerations

1. Scalability: The system is designed to handle increasing user traffic by scaling the number of pods in the Kubernetes cluster. This ensures that the application can perform efficiently under varying loads.

2. High Availability: Multiple replicas of the application are maintained using Kubernetes Deployments to ensure that the system remains available even if some pods fail.

3. Zero-Downtime Deployment: Rolling updates are implemented to allow seamless updates of the application without interrupting user access, ensuring continuous availability.

4. Fault Tolerance: Kubernetes automatically detects and replaces failed pods, ensuring system stability and minimizing downtime.

5. Containerization: Using Docker ensures consistency across different environments by packaging the application along with its dependencies.

6. Automation (CI/CD): Integration with CI/CD tools like GitHub Actions or Jenkins automates the build and deployment process, reducing manual errors and improving efficiency.

7. Load Balancing: Kubernetes Services distribute incoming traffic evenly across multiple pods, ensuring optimal resource utilization and performance.

8. Maintainability: The use of modular design and configuration files (YAML) makes the system easy to manage, update, and extend.

9. Portability: The containerized application can be deployed across different environments (local, cloud, hybrid) without major changes.

10. Security (Basic Level): Basic security practices such as controlled access, secure image usage, and environment isolation are considered in the design

2.6 API Catalogue

1. User Authentication API

- Endpoint: /api/auth/login
- Method: POST
- Description: Authenticates the user and returns an access token

2. Get Courses API

- Endpoint: /api/courses
- Method: GET
- Description: Retrieves the list of available courses

3. Get Course Details API

- Endpoint: /api/courses/{id}
- Method: GET
- Description: Fetches detailed information about a specific course

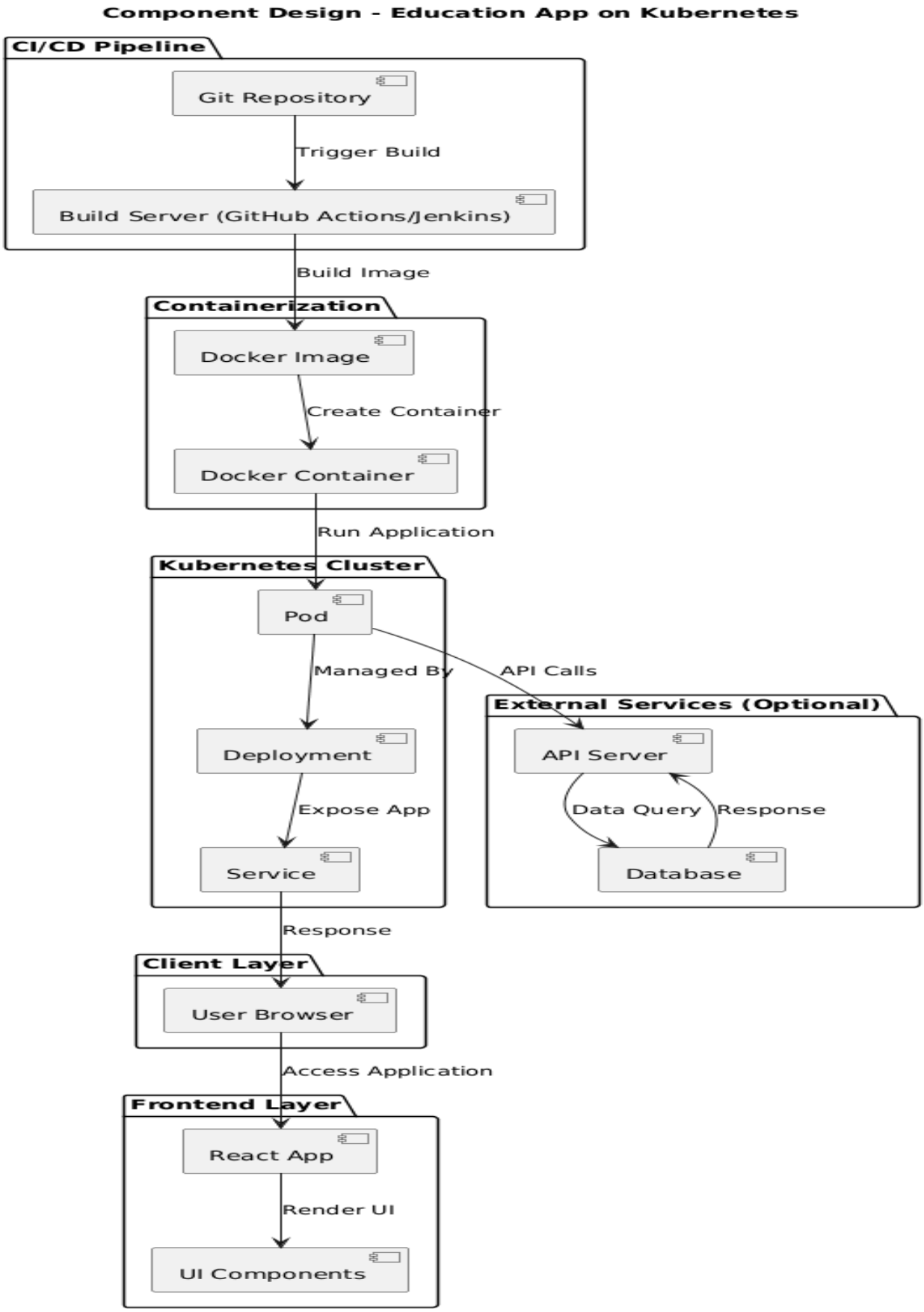
4. User Registration API

- Endpoint: /api/auth/register
- Method: POST
- Description: Registers a new user

5. Enroll in Course API

- Endpoint: /api/enroll
- Method: POST

- Description: Enrolls a user in a selected course



3. Data Design

Data design focuses on how data is structured, stored, and managed within the system to support application functionality. It defines the organization of data elements and their relationships to ensure efficient data handling and retrieval.

In this project, the data design is minimal as the current implementation is primarily frontend-based. However, it considers future integration with backend services, where data such as user information, course details, and enrollment records can be stored and managed. The design ensures scalability, consistency, and easy integration with databases in a containerized and Kubernetes-based environment.

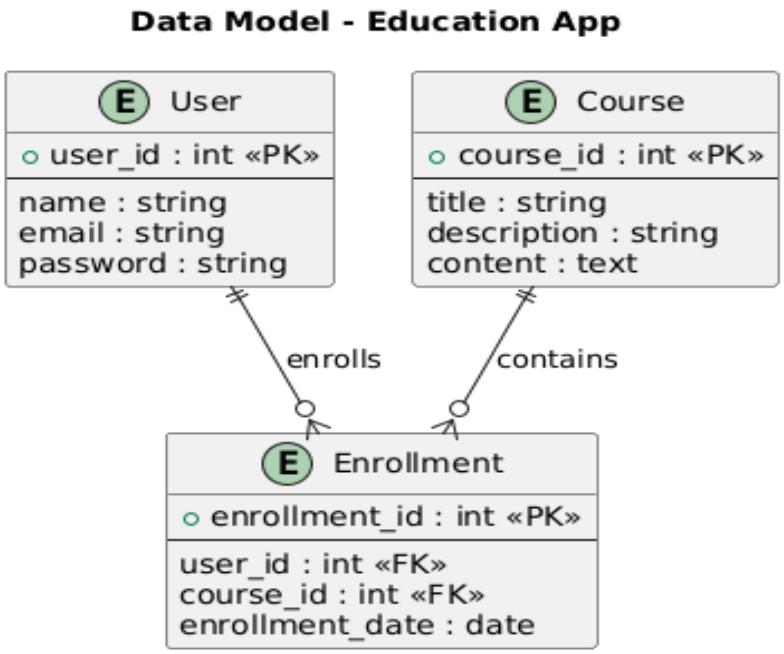
3.1 Data Model

The data model defines the logical structure of data used in the system, including entities, attributes, and their relationships. It provides a blueprint for how data is organized and managed within the application. In this project, the current implementation is primarily frontend-based, so a full-fledged database is not actively used.

However, a conceptual data model is designed to support future backend integration. The key entities identified include User, Course, and Enrollment.

- The User entity stores information such as user ID, name, email, and password.
- The Course entity contains details like course ID, title, description, and content.
- The Enrollment entity represents the relationship between users and courses, storing enrollment details such as user ID, course ID, and enrollment date.

The relationships are structured such that a user can enroll in multiple courses, and a course can have multiple users, forming a many-to-many relationship resolved through the Enrollment entity.



3.2 Data Access Mechanism

The data access mechanism defines how data is accessed, retrieved, and managed within the system. It ensures efficient communication between different components of the application while maintaining data consistency and performance.

In this project, the current implementation is primarily frontend-based, where data is either static or managed within the application state. However, the system is designed to support a scalable data access mechanism through backend APIs in future enhancements.

The React application interacts with backend services using HTTP/HTTPS requests. These requests are routed through Kubernetes Services, which act as a stable endpoint and distribute traffic across multiple pods running the application or backend services. This ensures load balancing and high availability.

When a user performs an action—such as viewing courses or enrolling—the frontend sends a request to the appropriate API endpoint. The backend processes the request, interacts with the database if required, and returns the response to the frontend. The frontend then updates the user interface accordingly.

Kubernetes plays a key role in ensuring reliable data access by:

- Routing requests to healthy pods
- Restarting failed pods automatically
- Maintaining service availability during updates

In future implementations, a dedicated data access layer (such as a service or API layer) can be introduced to handle database operations, enforce business logic, and improve security. Techniques such as authentication tokens, caching, and optimized queries can further enhance performance and data protection.

Overall, the data access mechanism is designed to be scalable, efficient, and adaptable to future backend integration while ensuring smooth communication across system components.

3.3 Data Retention Policies

Data retention policies define how long data is stored, how it is maintained, and when it is removed from the system. These policies help ensure efficient storage management, data privacy, and compliance with best practices. In this project, the current implementation is primarily frontend-based and does not store persistent user data. However, a conceptual data retention strategy is considered for future backend integration.

User Data:

- Stored as long as the user account is active
- Includes name, email, and authentication details

Course Data:

- Stored for long-term availability
- Includes course content, descriptions, and metadata

Enrollment Data:

- Retained to track user participation and course progress
- Helps in analytics and user history

Temporary Data:

- Includes session tokens and cached data
- Automatically expires after a defined time for security and performance

Logs and Monitoring Data:

- Kubernetes and application logs are stored for debugging and monitoring
- Retained for a limited duration and periodically deleted

Data Cleanup:

- Old or unnecessary data is removed to optimize storage
- Can include automated cleanup or archiving mechanisms

Security Considerations:

- Sensitive data should be securely stored and deleted when no longer needed
- Use of secure communication (HTTPS) and authentication mechanisms

Backup and Recovery:

- Data backup strategies can be implemented to prevent data loss
- Supports system recovery in case of failures

3.4 Data Migration

Data migration refers to the process of transferring data from one system, environment, or format to another. In this project, the current implementation does not involve active data migration as it is primarily frontend-based. However, migration strategies are considered for future backend integration and deployment environments.

Initial Data Setup:

- Static or sample data can be used during development
- Can be replaced with real data when backend services are introduced

Environment Migration:

- Data can be moved from local development (Minikube) to cloud-based Kubernetes environments
- Ensures consistency across different deployment stages

Database Migration (Future Scope):

- Schema creation and updates using migration scripts
- Version control for database changes

CI/CD Integration:

- Migration steps can be automated within the CI/CD pipeline
- Ensures smooth and consistent deployment across environments

Data Consistency:

- Validation checks should be performed before and after migration
- Prevents data loss or corruption

Backup Before Migration:

- Data should be backed up before performing any migration
- Enables rollback in case of failure

Zero-Downtime Migration:

- Strategies like rolling updates and versioning can be used
- Ensures application availability during migration

Security Considerations:

- Sensitive data should be encrypted during migration
- Access control should be enforced

4. Interfaces

1. User Interface (UI)

- Provides interaction between the user and the application
- Developed using React for a responsive and dynamic experience
- Allows users to view courses, navigate pages, and perform actions

2. API Interface

- Enables communication between frontend and backend services (future scope)
- Uses HTTP/HTTPS protocols for data exchange
- Supports operations such as fetching course data, user authentication, and enrollment

3. Kubernetes Interface

- Manages communication between application components inside the cluster
- Uses Kubernetes Services to expose applications and route traffic
- Ensures load balancing and high availability

4. Container Interface (Docker)

- Defines how the application runs inside containers
- Ensures consistent runtime environment across development and deployment
- Interacts with Kubernetes for container orchestration

5. CI/CD Interface

- Connects the code repository with build and deployment tools
- Automates processes such as building Docker images and deploying to Kubernetes
- Tools include GitHub Actions or Jenkins

6. External Interfaces (Optional/Future Scope)

- Includes third-party APIs or external services
- Can be used for authentication, analytics, or additional features
- Communicates securely using standard protocols

7. Network Interface

- Handles communication between different system components over the network
- Uses protocols like HTTP/HTTPS
- Ensures secure and reliable data transfer

5. State and Session Management

1. Application State Management

- The React application manages state using built-in hooks (e.g., useState, useEffect)
- State includes UI data such as selected courses, user inputs, and navigation status
- Ensures dynamic and responsive user interface updates

2. Global State Handling

- For larger applications, global state can be managed using tools like Context API or state management libraries (future scope)
- Helps in sharing data across multiple components efficiently

3. Session Management

- User sessions can be maintained using authentication tokens (e.g., JWT)
- Tokens are stored on the client side (e.g., local storage or session storage)
- Used to identify and authenticate users across requests

4. Stateless Communication

- The system follows a stateless communication model using HTTP/HTTPS
- Each request from the client contains all necessary information for processing

5. Kubernetes Role in State Management

- Kubernetes manages stateless application pods
- Ensures availability by restarting failed pods without affecting user sessions
- Session persistence can be handled externally if needed

6. Session Expiry and Security

- Sessions have a defined validity period
- Expired sessions require re-authentication
- Helps in maintaining system security

6. Caching

1. Frontend Caching

- The React application can cache data in browser memory or local storage
- Frequently accessed data such as course lists can be reused without repeated API calls
- Helps in faster UI rendering

2. API-Level Caching (Future Scope)

- Backend APIs can cache responses for commonly requested data
- Reduces repeated database queries
- Improves scalability and efficiency

3. Content Caching

- Static assets like images, CSS, and JavaScript files can be cached in the browser
- Reduces page load time and bandwidth usage

4. Kubernetes and Caching

- Kubernetes itself does not provide caching but supports integration with caching solutions
- Ensures availability of caching services by managing their deployment

5. Cache Expiration and Invalidation

- Cached data is stored for a limited time (TTL – Time To Live)
- Old or outdated data is automatically removed or refreshed
- Ensures data consistency

7. Non-Functional Requirements

7.1 Security Aspects

Authentication

- Ensures that only authorized users can access the system
- Can be implemented using secure login mechanisms
- Future scope includes token-based authentication (e.g., JWT)

Authorization

- Controls user access to different features and resources
- Ensures users can only perform permitted actions
- Role-based access control can be implemented in future

Secure Communication

- All data exchange should occur over HTTPS
- Prevents interception and tampering of data during transmission

Data Protection

- Sensitive user data (e.g., passwords) should be encrypted
- Avoid storing plain-text credentials
- Use secure storage practices

Container Security

- Use trusted and minimal Docker base images
- Regularly update images to fix vulnerabilities
- Avoid running containers with unnecessary privileges

Kubernetes Security

- Use namespaces to isolate resources
- Apply role-based access control (RBAC)
- Limit access to cluster resources

CI/CD Security

- Secure access to repositories and pipelines
- Protect API keys, tokens, and credentials using environment variables
- Avoid exposing sensitive information in code

Input Validation

- Validate user inputs to prevent common attacks
- Protect against issues like injection attacks

Session Security

- Use secure session handling mechanisms
- Implement session expiration and re-authentication
- Prevent unauthorized session access

Logging and Monitoring

- Monitor system activity for suspicious behavior
- Maintain logs for auditing and debugging
- Helps in detecting and responding to security incidents

7.2 Performance Aspects

Response Time

- The system should provide fast response to user requests
- UI interactions and page loads should occur with minimal delay

Scalability

- The system should handle increasing user load by scaling application instances (pods)
- Kubernetes supports horizontal scaling to maintain performance

Throughput

- The system should be capable of processing multiple user requests simultaneously
- Efficient handling of concurrent requests ensures better user experience

Resource Utilization

- Optimal use of CPU, memory, and network resources
- Kubernetes helps in managing and allocating resources efficiently

Load Balancing

- Incoming traffic is distributed evenly across multiple pods
- Prevents overloading of a single instance

Availability

- The system should remain operational with minimal downtime
- Kubernetes ensures high availability through multiple replicas

Efficient Deployment

- Rolling updates ensure that application updates do not affect performance
- No service interruption during deployment

Caching Support

- Frequently accessed data can be cached to reduce response time
- Reduces repeated processing and improves speed

Network Performance

- Efficient communication between components using optimized network routing
- Minimal latency in request and response flow

8. References

- Kubernetes Documentation. Available at: https://kubernetes.io/docs/
- Docker Documentation. Available at: https://docs.docker.com/
- React Documentation. Available at: https://react.dev/
- GitHub Actions Documentation. Available at: https://docs.github.com/en/actions
- Jenkins Documentation. Available at: https://www.jenkins.io/doc/
- Nginx Documentation. Available at: https://nginx.org/en/docs/
- Red Hat – Introduction to Kubernetes. Available at: https://www.redhat.com/en/topics/containers/what-is-kubernetes
- Google Cloud – Kubernetes Concepts. Available at: https://cloud.google.com/kubernetes-engine/docs/concepts
- Burns, B., Beda, J., Hightower, K. *Kubernetes: Up and Running*. O'Reilly Media, 2019.
- Merkel, D. *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014.